

Отчёт по лабораторной работе №4

Курс: Информатика, полиморфные абстрактные типы данных, 2 семестр.

Проект: sem2_lab4

Путь на MacBook Air: /Users/sfinks7802/CLionProjects/sem2_lab4

Постановка задачи

По техническому заданию нужно реализовать АТД LazySequence<T>, семейство потоков Stream<T>, выполнить тематическую задачу на их основе, покрыть алгоритмы модульными тестами и добавить пользовательский интерфейс. Также требуется ручной и автоматизированный режим тестирования, заранее подготовленные данные и проверки на больших объёмах.

Выбранная тематическая задача

Выбран вариант: сбор статистики потока событий в режиме онлайн.

Реализован класс OnlineStatistics<T>, который обрабатывает поток за один проход и считает:

- количество элементов;
- сумму;
- минимум и максимум;
- среднее значение;
- медиану;
- статистику по последнему окну: window min, window max, window mean.

Основные компоненты

LazySequence<T>

Файл: include/LazySequence.hpp

Реализованы:

- конечные и бесконечные ленивые последовательности;
- мемоизация материализованных элементов;
- Get, GetFirst, GetLast, GetSubsequence;
- Append, Prepend, InsertAt для элемента;
- Append, Prepend, InsertAt для Sequence<T>;
- RemoveAt;
- Concat;
- Map;
- Reduce;
- Where, включая ленивую фильтрацию бесконечной последовательности;
- Zip;
- GetLength и GetMaterializedCount;

- CreateGenerator.

Идея реализации: последовательность хранит уже вычисленный префикс. Если запрашивается ещё не материализованный индекс, элементы достраиваются по порождающему правилу и сохраняются. Повторный доступ к уже вычисленному элементу идёт напрямую в массив.

Generator<T>

Файл: include/LazySequence.hpp

Генератор инкапсулирует последовательное получение элементов:

- HasNext;
- GetNext;
- TryGetNext.

Также LazySequence<T> умеет создавать генератор через CreateGenerator().

Stream<T> и семейство потоков

Файл: include/Stream.hpp

Реализованы:

- Stream<T> как потоковая алгебра над LazySequence<T>;
- IReadStream<T>;
- IWriteOnlyStream<T>;
- ReadOnlyStream<T>;
- WriteOnlyStream<T>;
- ReadWriteStream<T>.

Поддерживаются:

- чтение из Sequence<T>;
- чтение из LazySequence<T> с ограничением count;
- чтение из строки;
- чтение из файла;
- запись в Sequence<T>;
- запись в файл;
- Read;
- Write;
- Seek;
- Open;
- Close;
- IsEndOfStream;
- GetPosition.

Пользовательский интерфейс

Консольный UI: src/lr4_console.cpp

Поддерживает:

- вывод первых N чисел Фибоначчи;
- Map для Фибоначчи;
- Where для конечного потока;
- ручное чтение потока из строки;
- ручную запись в поток;
- онлайн-статистику ручного потока;
- ручной выбор операции Map / Where / Reduce / Zip / Stats.

GTK UI: ui/lr4_gtk_main.cpp

Поддерживает:

- работу с Фибоначчи;
- Map, Where, Reduce, Zip;
- LazySequence subsequence;
- ручной поток;
- онлайн-статистику с медианой и оконным средним.

Соответствие требованиям ТЗ

Требование -> Реализация

1. Использовать Sequence<T> -> include/Sequence.hpp, ArraySequence<T>, ListSequence<T>.
2. Реализовать LazySequence<T> -> include/LazySequence.hpp.
3. Поддерживать бесконечные последовательности -> Cardinal::Infinity(), recurrence-конструктор, тесты с натуральным рядом.
4. Мемоизация -> EnsureIndexMaterialized, GetMaterializedCount.
5. Генератор -> Generator<T>, LazySequence::CreateGenerator().
6. Append / Prepend / InsertAt -> реализованы для элемента и Sequence<T>.
7. Remove -> RemoveAt.
8. Concat / Map / Reduce / Where / Zip -> реализованы в LazySequence<T> и Stream<T>.
9. Семейство потоков -> IReadOnlyStream, IWriteOnlyStream, ReadOnlyStream, WriteOnlyStream, ReadWriteStream.
10. Файлы и строки -> ReadOnlyStream / WriteOnlyStream с serializer/deserializer.
11. Ручное тестирование -> lr4_console, lr4_gtk.
12. Автоматизированное тестирование -> tests/lr4_tests.cpp.
13. Предельные значения -> тесты на 100000 lazy-элементов, файловый тест на 50000 чисел, benchmark на 1000000 lazy-элементов и файл 200000 чисел.
14. Тематическая задача -> OnlineStatistics<T>.

Тестирование

Файл: tests/lr4_tests.cpp

Результат последней проверки на MacBook Air:

- 38 passed;

- 0 failed.

Покрываются:

- базовые операции `Stream<T>`;
- `Map / Where / Reduce / Zip`;
- `LazySequence Get / GetFirst / GetLast / GetSubsequence`;
- `Concat`;
- бесконечный `Where`;
- `Append` к бесконечной последовательности без падения;
- `Generator`;
- чтение и запись потоков;
- файл `roundtrip`;
- `OnlineStatistics`;
- пустая статистика;
- `Append(Sequence*)` и `RemoveAt`;
- `ReadWriteStream`;
- негативные кейсы: чтение за концом, `bad Seek`, закрытый поток, ошибка парсинга;
- большой файловый поток;
- большой `lazy`-поток.

Benchmark

Файл: `src/lr4_benchmark.cpp`

Цель: `lr4_bench`

Benchmark выводит CSV:

`case,size,metric,value,ms`

Проверяются:

- `lazy_take` на 1000000 элементов;
- `file_write` на 200000 элементов;
- `file_read` на 200000 элементов;
- `online_stats`.

Пример результата на MacBook Air:

- `lazy_take,1000000,sum,499999500000;`
- `file_write,200000,items,200000;`
- `file_read,200000,sum,19999900000;`
- `online_stats,10,mean,14.3;`
- `online_stats,10,median,6.5;`
- `online_stats,10>window3mean,30.`

Команды запуска

Сборка и тесты:

`./build.sh`

Отдельный запуск тестов:

`./build/lr4_tests`

Консольный интерфейс:

`./build/lr4_console`

Графический интерфейс:

`./build/lr4_gtk`

Benchmark:

`./build/lr4_bench`

Пример ручного сценария

В `lr4_console`:

1. Выбрать пункт 7.
2. Ввести строку: 4 1 7 3 9 2
3. Выбрать операцию 5 Stats.

Ожидаемый результат:

`count=6 sum=26 min=1 max=9 mean=4.33333 median=3.5 window3mean=4.66667`

Вывод

Лабораторная работа реализует требуемые абстракции `LazySequence<T>` и семейство `Stream<T>`, содержит тематическую задачу онлайн-статистики, ручной и графический пользовательский интерфейс, автоматизированные тесты и стресс-проверки. Проект собирается через `build.sh`, тесты проходят успешно.